

2023.1 Multicore Computing
Midterm Exam (7pm-8:20pm, 4.21)

supervisor	
signature	

StudentID# : () , Name : ()

※ Your answer should be written in English.

1. (36 points) Fill in the blanks (a)~(r) with the most appropriate English words.

- (a.) is the capability of a system to handle a growing amount of work by adding resources to the system.
- (b.) means an undesirable situation that may occur when some possible interleaving of multiple threads can result in an undesired computation result.
- Threads belonging to the same process share (c.), (d.), and (e.).
- (f.) means ensuring that no two processes or threads are in their critical section at the same time.
- (g.) means federation of computer resources from multiple locations to reach a common goal. It tends to be more loosely (h.), heterogeneous, and geographically dispersed.
- (i.) problem describes two processes, the (j.) and the (k.), who share a common, (l.)-size buffer used as a queue. (same as (j)) generates a piece of data, put it into the buffer and start again. At the same time, (same as (k)) removes the data from the buffer one piece at a time. The problem is to make sure that the (same as (j)) won't try to add data into the buffer if it is (m.) and that the (same as (k)) won't try to remove data from an (n.) buffer. The solution for the (same as (j)) is to go to sleep if the buffer is (same as (m)). The next time the (same as (k)) removes an item from the buffer, it notifies the (same as (j)), who starts to fill the buffer again. In the same way, the (same as (k)) can go to sleep if it finds the buffer to be (same as (n)). The next time the (same as (j)) puts data into the buffer, it wakes up the sleeping (same as (k)).
- A barrier for a group of threads or processes in the source code means any thread/process must (o.) at this point and cannot (p.) until all other threads/processes (q.) it.
- Lock (r.) means threads can acquire a lock in the order they request it.

2. (9 points) (a) In JAVA, the code "**Semaphore x = new Semaphore(N);**" can be used for creating a Semaphore object. What is the meaning of **N** in the code? What is the purpose of using **N** in the code? Explain with sufficient details.
()

(b) In JAVA, **ArrayBlockingQueue** is a bounded, blocking queue that stores elements internally in an array.

(i) Here, "bounded" means what? ()

(ii) Here, "blocking" means what? ()

Your answer should contain what happens when.

3. (12 points) Fill in following blanks (a)~(d) with the most appropriate words

Comparison between fine grain system and coarse grain system	
fine grain system	coarse grain system
low (a.) ratio	high (same as (a)) ratio
relatively small amounts of (b.)	relatively large amounts of (same as (b))
disadvantage: relatively high (c.) overhead	advantage: relatively low (same as (c)) overhead
advantage: relatively easier to (d.)	disadvantage: relatively harder to (same as (d))

4.(43 points) (a) Following multi-threaded java codes compute and display the sum of each element in an array with size **NUM_END** (**int_arr**) that is initialized as {1, 2, 3,..., **NUM_END**}. (Assume **NUM_END** is divisible by **NUM_THREAD**.) There can be two approaches, a naive approach and a divide-and-conquer approach, we learned in our lectures. In the naive approach, the entire array is divided into **NUM_THREAD** sub-arrays and each thread simultaneously computes the sum of its assigned sub-array. The main thread adds together each thread's answer for the final result. In the divide-and-conquer approach, a thread recursively creates two threads (left and right) to compute the sum of left half and right half of the array, respectively until reaching at some point. Fill in empty boxes below with appropriate JAVA codes for the naive approach and divide-and conquer approach..

-----< source code >-----

< Naive Approach>

```
class SumThread extends Thread {
    int lo; // fields for communicating inputs
    int hi;
    int[] arr;
    int ans = 0; // for communicating result
    SumThread(int[] a,int l,int h) { lo=l; hi=h; arr=a; }
    public void run() {
        // insert your JAVA code here
    }
}

class ex2 {
    private static int NUM_END=10000;
    private static int NUM_THREAD=4;

    public static void main(String[] args) {
        int[] int_arr = new int [NUM_END];
        int i,s;
        for (i=0;i<NUM_END;i++) int_arr[i]=i+1;
        s=sum(int_arr);
        System.out.println(s);
    }

    static int sum(int[] arr) {
        // insert your JAVA code here
    }

    return ans;
}
}
```

< Divide-and-Conquer (Recursive) Approach >

```
class SumThread extends Thread {
    int lo; // fields for communicating inputs
    int hi;
    int[] arr;
    int ans = 0; // for communicating result
    private static int SEQUENTIAL_CUTOFF=100;
    SumThread(int[] a,int l,int h) { lo=l; hi=h; arr=a; }
    public void run() {
        // insert your JAVA code here
    }
}

class ex2 {
    private static int NUM_END=10000;

    public static void main(String[] args) {
        int[] int_arr = new int [NUM_END];
        int i,s;
        for (i=0;i<NUM_END;i++) int_arr[i]=i+1;
        s=sum(int_arr);
        System.out.println(s);
    }

    static int sum(int[] arr) {
        // insert your JAVA code here
    }

    return ans;
}
}
```

- (b) What is the main problem (drawback) of the naive approach? Why does such problem occur?. Explain 'what' and 'why'.
 (i) what: ([use less than 10 English words])
 (ii) why: ([use less than 20 English words])
- (c) What is the meaning of SEQUENTIAL_CUTOFF in the divide-and-conquer code? Why is it necessary? Explain 'what' and 'why'.
 (i) what: ([use less than 10 English words])
 (ii) why: ([use less than 20 English words])
- (d) Explain what problem may occur if SEQUENTIAL_CUTOFF is too small in the divide-and-conquer code?
 ([use less than 20 English words])